

Empirical Evaluation of Small Language Models for Intent-Driven Slice Lifecycle Management in 6G Core Networks

A Systematic Study of Model Architecture, Size Scaling, and Quantization Impact on CPU-Only Edge Inference

[Authors — RB-RU Research]

July 2026

Abstract

Intent-driven management is a cornerstone of 6G network automation, enabling operators to express desired outcomes in natural language that autonomous agents translate into orchestration commands. Deploying such agents at the network edge on resource-constrained, CPU-only infrastructure demands Small Language Models (SLMs) that balance accuracy with inference speed, memory footprint, and energy consumption. This paper presents a comprehensive empirical evaluation of open-weight SLMs for 6G intent-to-action translation across two controlled experiments with per-intent resource monitoring. Phase 1 benchmarks five model families at the ~8B scale (Llama 3.1 8B, Qwen 2.5 7B, Gemma 2 9B, GLM-4 9B, and DeepSeek-R1 8B) under identical Q4_K_M quantization, hardware, and prompt conditions on 200 real-world intents spanning three slice management domains. Phase 2 extends the winning architecture (Qwen 2.5) across four model sizes (0.5B to 7B) and three K-quant levels (Q2_K, Q4_K_M, Q8_0) in a full-factorial 12-point design, quantifying accuracy, inference speed, throughput, CPU utilization, memory footprint, energy consumption, domain robustness, and complexity resilience — all with 95% confidence intervals over 200 intents per configuration. Key findings include: (1) Qwen 2.5 7B at Q4_K_M achieves the best accuracy–efficiency balance (29.8% exact match, 15.0 s/intent), outperforming Gemma 2 9B by 2.7x in speed at equivalent accuracy; (2) measured CPU utilization scales with model size — from 69% at 0.5B to 94% at 7B — empirically demonstrating that memory bandwidth, not compute, is the binding constraint of CPU inference, and that flat-utilization energy models overestimate small-model energy by 15–25%; (3) aggressive 2-bit quantization of larger models (7B Q2_K, 29.0% exact) surpasses 8-bit quantization of smaller models (3B Q8_0, 26.8%); (4) diminishing returns manifest in both model size (3B achieves 92% of 7B accuracy at 60% of its energy) and quantization precision (Q4-to-Q8 aperture below 1 percentage point at all sizes); (5) a quantization–speed crossover exists: Q8_0 is the fastest variant at 1.5B–3B but the slowest at 7B, where its 11.4 GB working set saturates memory bandwidth; (6) Slicing Provisioning emerges as a solved domain (100% action accuracy at 3B and above) while Conflict Resolution remains near-random (5–13%), underscoring the reasoning gap in current SLMs; and (7) reasoning-oriented architectures (DeepSeek-R1) are fundamentally incompatible with CPU-only inference, producing unbounded token generation. We contribute the first systematic cross-family, multi-size, multi-quantization SLM benchmark for 6G intent-driven slice management with real per-intent resource telemetry, a detailed Pareto analysis spanning accuracy, speed, energy, and memory dimensions, and actionable deployment guidelines for resource-constrained 6G edge environments.

1. Introduction

1.1 Motivation

The transition to sixth-generation (6G) mobile networks elevates intent-driven management from an aspirational concept to a core operational paradigm [1]. In this model, network operators articulate desired outcomes in natural language—such as “provision a URLLC slice with sub-millisecond latency for autonomous vehicle platooning”—and autonomous AI agents translate these intents into executable network configuration commands. This approach promises to

dramatically reduce operational complexity, accelerate service provisioning, and enable zero-touch automation at scale [2, 3].

However, the architectural shift toward distributed, disaggregated 6G core networks introduces a fundamental tension. While Large Language Models (LLMs) like GPT-4 and Llama-70B have demonstrated impressive intent understanding [4, 5], their deployment at the network edge is constrained by three hard realities of telco infrastructure: (1) **latency budgets**—URLLC slices demand sub-millisecond control loops incompatible with cloud-roundtrip inference; (2) **energy constraints**—edge nodes operate under strict power envelopes, often without GPU accelerators; and (3) **hardware heterogeneity**—6G infrastructure spans from centralized data centers to far-edge compute nodes with as little as 4–8 CPU cores and a few gigabytes of RAM [6].

Small Language Models (SLMs)—models under 10 billion parameters—emerge as a compelling middle ground. Recent advances in architecture design (Llama 3.1 [7], Qwen 2.5 [8], Gemma 2 [9], GLM-4 [10]) and post-training quantization [11, 12] have produced open-weight SLMs that approach LLM-quality performance on structured tasks while fitting within edge hardware constraints. Yet systematic evaluations comparing these models on domain-specific network orchestration tasks remain absent from the literature.

1.2 Research Gaps

Existing work on SLM evaluation suffers from four limitations when applied to 6G intent-driven management:

1. **Single-family bias:** Most benchmarks evaluate one model family in isolation, precluding cross-architecture comparisons that are essential for technology selection in real deployments [13].
2. **Generic benchmarks:** Standard NLP benchmarks (MMLU, HellaSwag, GSM8K) measure broad language understanding but do not capture the structured output generation and domain-specific reasoning required for network intent translation [12, 14].
3. **Neglect of quantization interaction:** While quantization impact on perplexity is well-studied, its interaction with model size on *structured task accuracy* (generating valid JSON network commands) has not been characterized. A model that preserves perplexity under aggressive quantization may still fail to produce syntactically correct JSON [15, 16].

4. **Absence of resource telemetry:** Prior SLM benchmarks report accuracy and latency but rarely measure CPU utilization, resident memory, or energy per inference on the actual deployment hardware class. Capacity planning and sustainability assessment for 6G edge nodes require these measurements, not TDP-based extrapolations.

1.3 Contributions

This paper addresses these gaps through two rigorously controlled experiments and a multi-dimensional analysis:

- **Phase 1—Cross-Family Comparison (Section 4):** We benchmark five open-weight model families at the ~8B parameter scale under identical conditions: same quantization (Q4_K_M), identical hardware (16-core CPU VM, 31 GB RAM), uniform prompt design, and a shared dataset of 200 real-world 6G intents. We report accuracy, speed, domain-level breakdowns, complexity robustness, and token efficiency.
- **Phase 2—Size Scaling and Quantization Impact (Section 5):** Selecting the winning architecture (Qwen 2.5), we conduct a full-factorial evaluation of 12 configurations: 4 model sizes (0.5B, 1.5B, 3B, 7B) x 3 K-quant levels (Q2_K, Q4_K_M, Q8_0). This design isolates the independent effects of parameter count and weight precision, revealing non-obvious interactions and cross-size Pareto dominance relationships.
- **Per-Intent Resource Telemetry with Statistical Rigor (Section 5.8):** Every Phase 2 inference is instrumented with 0.5-second psutil sampling of system-wide CPU utilization and inference-process resident memory, from which per-intent energy is derived using measured utilization (not assumed TDP fractions). All metrics are reported as mean $\pm$ 95% confidence interval (t-distribution, $n = 200$), enabling principled comparison across configurations.
- **Comprehensive Multi-Metric Analysis:** Beyond aggregate accuracy and speed, we provide domain-level (Slicing Provisioning, Scaling Request, Conflict Resolution), complexity-level (Simple, Complex, Ambiguous), token efficiency, memory footprint, and energy efficiency analyses for all 16 evaluated configurations.
- **Actionable Deployment Guidelines (Section 6):** We synthesize findings into concrete recommendations for SLM selection and configuration across

diverse 6G edge deployment scenarios, from ultra-low-latency far-edge nodes to capacity-rich regional edge servers.

2. Related Work

2.1 Intent-Based Networking in 5G/6G

Intent-Based Networking (IBN) has been formalized as a key enabler for zero-touch network and service management (ZSM) by ETSI [2] and incorporated into 3GPP Release 18 specifications for network slice lifecycle operations [3]. The core premise is that operators express *what* they want (intent) rather than *how* to achieve it (low-level configuration), with an automated translation layer mapping intents to network commands. Recent work has explored LLM-based intent translation [4, 5], but the computational demands of large models conflict with the distributed, latency-sensitive nature of 6G edge deployments.

2.2 Small Language Models for Structured Tasks

The 2024–2025 period has seen a proliferation of high-quality open-weight SLMs. Llama 3.1 from Meta [7] demonstrated that 8B-parameter models can approach the performance of much larger predecessors on reasoning benchmarks. Qwen 2.5 from Alibaba [8] introduced architectural improvements yielding strong performance on code and structured output tasks across a size range from 0.5B to 72B. Google’s Gemma 2 [9] applied knowledge distillation to produce competitive 9B models, while Tsinghua’s GLM-4 [10] explored bilingual capabilities. Concurrent surveys by Jiang et al. [13] and Fan et al. [14] have catalogued the SLM landscape, but neither includes domain-specific network orchestration evaluations.

2.3 Post-Training Quantization

Post-training quantization (PTQ) compresses model weights to lower bit precision, reducing memory footprint and inference latency [11]. The GGUF K-quants family [12]—spanning Q2_K (2-bit) through Q8_0 (8-bit)—has become the de facto standard for CPU-based LLM inference via the llama.cpp ecosystem. Recent studies by Dettmers et al. [15] and Xiao et al. [16] have characterized quantization accuracy degradation on standard NLP metrics, but domain-specific structured tasks may exhibit different sensitivity profiles. No prior work has systematically evaluated the three-way interaction among model architecture,

parameter count, and quantization precision on structured intent-to-action translation.

3. Methodology

3.1 Dataset Construction

We construct a dataset of **200 intents** spanning three 6G network slice management domains, each annotated with a ground-truth JSON action:

Domain	Count	Description	Example
Slicing Provisioning	70	Creation of new network slices with QoS parameters	“Create a URLLC slice with 1ms latency for autonomous vehicle platooning in Bangkok”
Scaling Request	70	Modification of existing slice capacity	“Scale the eMBB slice for stadium event to 10 Gbps during the concert”
Conflict Resolution	60	Resolution of resource conflicts between competing slices	“URLLC factory slice needs priority over eMBB streaming slice during emergency”

Each intent is assigned a complexity label: **Simple** (explicit, single-domain parameters), **Complex** (multi-constraint or regional context), or **Ambiguous** (implicit priorities requiring contextual inference). This three-way split enables evaluation of robustness across the difficulty spectrum.

The dataset bridges synthetic generation (to ensure coverage of edge cases) with real-world 6G deployment scenarios validated by domain experts.

3.2 Models Evaluated

Phase 1 evaluates five model families at the ~8B parameter scale. All models use **Q4_K_M** quantization (4-bit K-quant, medium variant), the most widely deployed default in the llama.cpp ecosystem, to ensure fair comparison:

#	Model	Ollama Tag	Family	Params	Quant	Disk	Source
1	Llama 3.1 8B	llama3.1:8b	Meta (USA)	8.0B	Q4_K_M	4.9 GB	Ollama library
2	Qwen 2.5 7B	qwen2.5:7b	Alibaba (CN)	7.6B	Q4_K_M	4.7 GB	Ollama library
3	Gemma 2 9B	gemma2:9b-q4_K_M	Google (USA)	9.4B	Q4_K_M	5.4 GB	Custom GGUF
4	GLM-4 9B	glm4:9b-q4_K_M	Tsinghua (CN)	9.4B	Q4_K_M	5.5 GB	Custom GGUF
5	DeepSeek-R1 8B	deepseek-r1:8b	DeepSeek (CN)	8.0B	Q4_K_M	5.2 GB	Ollama library

Critical note: The default Ollama tags for Gemma 2 and GLM-4 use Q4_0 quantization, which differs from Q4_K_M in block structure and compression ratio. To ensure fair comparison, we downloaded the corresponding Q4_K_M GGUF files from HuggingFace (bartowski/gemma-2-9b-it-GGUF and bartowski/glm-4-9b-chat-GGUF) and created custom Ollama models via Modelfile. This attention to quantization parity is essential for valid cross-family inference.

Phase 2 evaluates Qwen 2.5—the Phase 1 winner—in a full-factorial design: 4 model sizes x 3 quantization levels = 12 configurations:

Size	Params	Q2_K (2-bit)	Q4_K_M (4-bit)	Q8_0 (8-bit)
0.5B	494M	338 MB	397 MB	531 MB
1.5B	1.54B	~650 MB	986 MB	~1.6 GB
3.0B	3.09B	1.3 GB	1.9 GB	3.1 GB
7.0B	7.62B	~4.0 GB	4.7 GB	~7.5 GB

All Q2_K and Q8_0 variants were built from HuggingFace GGUF files (bartowski/Qwen2.5-{size}-Instruct-GGUF) with identical Modelfile templates, eliminating any confounding from metadata or chat template differences.

3.3 Infrastructure

All experiments execute on two identical virtual machines provisioned on a Proxmox VE 8.2 hypervisor (host: 172.16.1.206):

VM	VMID	IP	CPU	RAM	Disk	OS
lab-slm-01	206	172.16.1.213	16 vCPU	31 GiB	80 GB	Ubuntu 24.04
lab-slm-02	207	172.16.1.214	16 vCPU	31 GiB	80 GB	Ubuntu 24.04

Each VM runs **Ollama 0.5.x** as the inference server, configured for CPU-only inference with no GPU acceleration. The 16-core allocation was empirically determined: an initial 8-core configuration left CPU utilization at 50–60%, indicating underutilization. Our per-intent telemetry (Section 5.8) subsequently revealed that utilization at 16 cores is itself size-dependent (69–94%), a finding with direct energy-modeling implications.

Workload distribution across the two VMs was parallelized to minimize total experiment wall-clock time: lab-slm-01 executed the nine 0.5B–3B configurations and lab-slm-02 the three 7B configurations.

3.4 Inference Protocol and Resource Instrumentation

All experiments follow a uniform protocol implemented in `benchmark_runner_v2.py`:

1. **System Prompt:** A fixed enriched prompt instructs the model: “You are a 6G Core Network Slice Management Agent. Your job is to translate natural language intents into JSON commands for network orchestration.” Rules mandate JSON-only output, no markdown, no explanations, with a one-shot example.
2. **Inference Parameters:** `temperature=0.1` (minimal stochasticity), `num_predict=1024` (sufficient for JSON output), `stream=false` (batch mode).
3. **Warmup:** Two warmup inferences before timing to initialize model weights in memory.

4. **No Retry Sampling:** Each intent is evaluated exactly once. This is intentional: our benchmark measures *reliability*, not best-of-N sampling performance, which is more representative of production deployment constraints.
5. **Resource Sampling:** During each inference, a background thread samples
 - (a) system-wide CPU utilization via `psutil.cpu_percent()` and (b) the resident set size (RSS) of the llama-server inference process, at 0.5-second intervals. The psutil counter is primed before each sampling loop to avoid the documented first-call-returns-zero artifact. Per-intent energy is derived as $E = (\text{mean CPU utilization fraction}) \times \text{TDP} \times (\text{inference time})$, with TDP = 135 W for the 16-core Xeon E5-2690 v4 allocation.

Statistical framework: For every continuous metric we report the mean, standard deviation, and 95% confidence interval computed with the t-distribution ($df = 199$, $t \approx 1.97$) over the 200 intents of each configuration. CI half-widths are typically 5–12% of the mean, confirming that the observed cross-configuration differences discussed below are not sampling artifacts.

Note on DeepSeek-R1: For the reasoning model, the Ollama API returns both a thinking field (chain-of-thought) and a content field (final answer). Our benchmark runner joins both fields and strips `<think>...</think>` tags before JSON parsing. Despite this post-processing, the model exhibited pathological behavior on CPU (see Section 4.4).

3.5 Evaluation Metrics

We report seven categories of metrics:

Metric	Definition	Relevance
Format Accuracy	% of responses parsing as valid JSON with an action field	Measures output compliance; non-compliant responses are useless in automated pipelines
Action Correct	% where the predicted action matches ground truth exactly	Core task accuracy for the primary output field
Exact Match	Average % of ground-truth key–value pairs correctly reproduced	Holistic accuracy capturing both action and parameter fidelity

Metric	Definition	Relevance
Inference Time	Wall-clock seconds per intent (mean $\pm$ 95% CI)	Latency budget compliance for edge deployment
Throughput	Tokens generated per second	Computational efficiency
CPU / Memory	System-wide CPU % and peak process RSS (psutil, 0.5s sampling)	Capacity planning; reveals the bandwidth bottleneck
Energy	Measured-utilization x TDP x time, joules per intent	Sustainability and operating cost

Additionally, we report **Total Tokens**, **Total Benchmark Time**, and **Domain-Level Breakdowns** for all configurations.

3.6 Prompt Design

The system prompt used across all experiments:

You are a 6G Core Network Slice Management Agent.
Your job is to translate natural language intents into JSON commands for network orchestration.

Rules:

1. Respond ONLY with a valid JSON object---no markdown, no explanations, no code fences
2. The JSON MUST contain an "action" field
3. Use these slice types when appropriate: "eMBB", "URLLC", "mMTC"
4. For ambiguous requests, infer the best configuration based on context
5. Include all relevant parameters: latency_ms, bandwidth_gbps, device_density, priority

Example:

Input: "Create a URLLC slice with 1ms latency for robot control"

Output: {"action": "CREATE", "slice_type": "URLLC", "latency_ms": 1}

This prompt design was validated in prior work [17] and shown to substantially improve format compliance over minimal prompts.

4. Phase 1: Cross-Family Model Comparison

Phase 1 addresses the question: *Which open-weight SLM family performs best for 6G intent translation at the ~8B parameter scale under identical conditions?*

4.1 Aggregate Performance

Table 1 presents the headline results. The Qwen 2.5 7B row reflects the instrumented re-run under the v2 protocol (identical scoring); the remaining rows are from the original v1 campaign under the same prompt, dataset, and hardware.

Table 1: Phase 1—Cross-Family Performance at ~8B Parameters (Q4_K_M)

Ran k	Mode l	Format (%)	Action (%)	Exact (%)	Time (s)	Tok/s	Total Tokens
1	Qwen 2.5 7B	100.0	42.5	29.8	15.0	2.8	8,343
2	Gemma 2 9B	98.5	45.5	29.3	39.8	0.99	8,028
3	Llama 3.1 8B	100.0	36.5	26.0	19.1	2.21	8,452
4	GLM-4 9B	100.0	29.0	24.6	22.9	1.78	8,130
5	DeepSeek-R1 8B	0.0*	0.0*	0.0*	77.9*	3.29	51,200

DeepSeek-R1 results reflect pathological behavior on CPU; see Section 4.4 for detailed analysis.

Key observations:

- **Qwen 2.5 7B dominates the Pareto frontier:** It achieves the highest exact match (29.8%, 95% CI [25.8, 33.9]) while maintaining the fastest inference among models with >95% format accuracy (15.0 s/intent, CI [13.9, 16.1]). This represents a 2.7x speed advantage over Gemma 2 9B at equivalent accuracy.

- **Gemma 2 9B provides marginal action prediction advantage** (45.5% vs. 42.5%) but at prohibitive latency cost (39.8 s/intent). This trade-off may be acceptable for non-real-time batch processing but is unsuitable for interactive intent management.
- **Llama 3.1 8B offers a balanced profile** with perfect format compliance and competitive speed (19.1 s/intent), though exact match trails Qwen by 3.8 percentage points. Its action accuracy (36.5%) places it between Qwen and GLM-4.
- **GLM-4 9B underperforms** across all accuracy dimensions despite comparable model size and quantization. This may indicate suboptimal alignment with JSON-structured output or English-language prompts, as GLM-4’s training emphasizes bilingual Chinese-English capabilities.
- **Format accuracy ceiling:** Three of four non-reasoning models achieve perfect (100.0%) format compliance, and Gemma 2 9B reaches 98.5%. This demonstrates that the enriched prompt design effectively constrains SLM output structure at the ~8B scale, addressing a common failure mode in open-ended generation.

4.2 Domain-Level Decomposition

Intent difficulty varies dramatically by domain. **Table 2** decomposes Action Correct by domain:

Table 2: Phase 1—Action Correct by Domain (%)

Model	Provisioning (n=70)	Scaling (n=70)	Conflict (n=60)
Qwen 2.5 7B	100.0	18.6	6.7
Gemma 2 9B	97.1	25.7	8.3
Llama 3.1 8B	87.1	11.4	6.7
GLM-4 9B	54.3	21.4	8.3
Mean across 4 models	84.6	19.3	7.5

Three distinct difficulty tiers emerge:

1. **Slicing Provisioning (solved):** Mean action accuracy of 84.6% across families. Qwen achieves perfect 100% accuracy, and even the weakest

model (GLM-4) reaches 54.3%. The CREATE action with explicit QoS parameters maps cleanly to model capabilities.

- 2. **Scaling Request (challenging):** Mean 19.3%. Models must interpret scaling direction (up/down), quantify capacity changes, and understand temporal context (“during the concert”). Gemma 2 leads at 25.7%, suggesting its training data may include more parameter-modification patterns.
- 3. **Conflict Resolution (near-random):** Mean 7.5%, approaching the baseline of random guessing among multiple action types. This domain requires multi-constraint reasoning (evaluate two competing requests, determine priority, resolve resource contention)—capabilities that current SLMs at the ~8B scale demonstrably lack.

The domain difficulty hierarchy (Provisioning >> Scaling >> Conflict) is robust across all model families, indicating that it stems from inherent task complexity rather than model-specific failures.

4.3 Complexity Robustness

Table 3 examines how models degrade when moving from Simple to Complex intents.

Table 3: Phase 1—Exact Match by Complexity (%)

Model	Simple	Complex	Degradation (pp)
Qwen 2.5 7B	46.7	20.4	-26.3
Gemma 2 9B	45.3	19.9	-25.4
Llama 3.1 8B	40.3	17.5	-22.8
GLM-4 9B	40.5	16.2	-24.3

The complexity gap is stark and consistent: exact match drops by 22.8–26.3 percentage points from Simple to Complex intents across all families. This represents a fundamental limitation of current SLMs: they handle explicit, single-parameter intents competently but degrade sharply when intents require contextual inference, implicit priority resolution, or multi-constraint reasoning.

4.4 DeepSeek-R1: Diagnostic Case Study

DeepSeek-R1 8B, a reasoning-oriented model employing chain-of-thought generation, produced 0.0% format accuracy across all 200 intents despite

generating 51,200 total tokens—6.2x more tokens than the next highest model (Llama 3.1, 8,452 tokens). This pathological behavior warrants detailed examination as it highlights a critical deployment consideration for 6G edge environments.

Root cause analysis:

1. **Unbounded reasoning:** DeepSeek-R1’s architecture generates thinking tokens (chain-of-thought) before producing a final answer. On CPU with a 1024-token `num_predict` limit, the model exhausts the token budget during the reasoning phase, never reaching the answer-generation stage. This results in responses containing only partial reasoning traces with no extractable JSON.
2. **Ollama API structure:** The Ollama API separates reasoning model output into `message.thinking` and `message.content` fields. Our initial benchmark runner read only content, which was empty for all intents. A subsequent fix joined both fields and stripped `<think>` tags, but the underlying issue—that valid JSON is never generated within the token budget—persisted.
3. **Infinite generation on CPU:** Even with relaxed token limits, the model exhibited unbounded thinking loops on CPU-only inference. A simple “Hello” test timed out after 10 seconds without producing output, with the llama-server process consuming 1,556% CPU across 16 cores for over 8 hours before manual termination. This behavior is not observed on GPU-accelerated inference, suggesting that the slower per-token generation rate on CPU interacts pathologically with the model’s reasoning termination criteria.

Implications for 6G deployment: Reasoning-oriented architectures are currently incompatible with CPU-only edge inference for latency-bounded applications. Until robust token budgeting or early termination mechanisms are developed for reasoning models on CPU, standard autoregressive architectures remain the only viable option for 6G edge SLM deployment. This finding is itself a contribution: the SLM community’s rush toward reasoning-enhanced architectures must be tempered by deployment reality checks on resource-constrained hardware.

4.5 Token Efficiency

Total tokens generated per benchmark run is a proxy for both computational cost and latency. Across non-reasoning Phase 1 models, token counts are remarkably

consistent: 8,028–8,452 tokens for 200 intents (40–42 tokens/intent). This uniformity suggests that model architecture affects *what* tokens are generated (content quality) more than *how many* tokens are generated (verbosity).

DeepSeek-R1’s 51,200 tokens (256 tokens/intent) represents a 6.2x overhead with zero useful output—a catastrophic efficiency failure for edge deployment.

5. Phase 2: Size Scaling and Quantization Impact

Phase 2 addresses two interconnected questions: (1) *How does Qwen 2.5 accuracy scale with model size from 0.5B to 7B?* and (2) *How does quantization precision (Q2_K, Q4_K_M, Q8_0) affect the accuracy–efficiency trade-off at each size?* All 12 configurations were executed under the instrumented v2 protocol (Section 3.4) with per-intent CPU, memory, and energy telemetry; every value below carries a 95% confidence interval over n = 200 intents.

5.1 Accuracy Scaling

Table 4 presents exact match for all 12 configurations.

Table 4: Phase 2—Exact Match (%) by Model Size and Quantization Level [95% CI]

Size	Q2_K (2-bit)	Q4_K_M (4-bit)	Q8_0 (8-bit)	Q2-to-Q8 Delta	Q4-to-Q8 Gain
0.5B	18.3 [14.9, 21.8]	18.5 [15.0, 22.0]	19.4 [16.0, 22.8]	+1.1	+0.9
1.5B	22.1 [18.4, 25.7]	25.8 [21.9, 29.7]	25.7 [21.6, 29.7]	+3.6	-0.1
3.0B	24.1 [20.3, 28.0]	27.3 [23.2, 31.4]	26.8 [22.7, 30.9]	+2.7	-0.5
7.0B	29.0 [24.9, 33.0]	29.8 [25.8, 33.9]	30.5 [26.3, 34.6]	+1.5	+0.7
Me an Delta	—	—	—	+2.2	+0.3

Finding 1: Diminishing returns in model size. The marginal accuracy gain per parameter doubling decreases sharply (at Q4_K_M): - 0.5B to 1.5B (3x params): +7.3 pp - 1.5B to 3.0B (2x params): +1.5 pp - 3.0B to 7.0B (2.3x params): +2.5 pp

The 3B model at Q4_K_M achieves **91.6% of 7B accuracy** (27.3% vs. 29.8%) while occupying only **40.4% of the disk footprint** (1.9 GB vs. 4.7 GB), consuming **60% of the energy** (1,122 J vs. 1,884 J per intent), and running **1.6x faster** (9.4 s vs. 15.0 s). This marks 3B Q4_K_M as the *efficiency sweet spot* for deployment scenarios where memory, energy, or latency budgets preclude 7B models.

Finding 2: Quantization sensitivity peaks at intermediate sizes. The Q2-to-Q8 accuracy gap—a measure of quantization sensitivity—is non-monotonic: - **0.5B**: only 1.1 pp degradation (robust to aggressive quantization) - **1.5B**: 3.6 pp degradation (peak sensitivity) - **3.0B**: 2.7 pp degradation (moderate) - **7.0B**: 1.5 pp degradation (robust again)

This inverted-U sensitivity curve suggests competing effects: at very small sizes, model capacity is so limited that quantization precision matters little (the model already operates near its architectural ceiling). At 1.5B, the model has sufficient capacity to benefit from higher precision, making it sensitive to quantization. At 7B, the model’s abundant capacity provides redundancy that masks weight perturbation, restoring robustness. The deployment implication: when quantizing aggressively (Q2_K), prefer 0.5B or 7B over intermediate sizes.

Finding 3: Q4-to-Q8 provides negligible gain. Across all sizes, the accuracy gap between Q4_K_M and Q8_0 averages only 0.3 percentage points (range: -0.5 to +0.9), well inside every configuration’s confidence interval. The negative gains at 1.5B and 3B are within measurement noise but consistent with the broader pattern: Q4_K_M is the *quantization sweet spot*—statistically indistinguishable from Q8_0 at substantially smaller size (30–55% smaller on disk, ~50% smaller in RAM).

Finding 4: Cross-size Pareto dominance. The 7B Q2_K configuration (29.0% exact) outperforms the 3B Q8_0 configuration (26.8%) in accuracy while occupying comparable disk space (~4.0 GB vs. 3.1 GB). This challenges the intuitive heuristic that higher-bit quantization of a smaller model is always preferable to lower-bit quantization of a larger model. A 7B model at Q2_K provides near-7B accuracy at a footprint closer to 3B models, enabling deployment on edge nodes that would otherwise be forced to use smaller architectures—though at roughly 1.8x the inference latency of 3B Q8_0 (13.8 s vs. 7.7 s), a trade-off quantified fully in Section 5.8.

5.2 Inference Efficiency

Table 5 reports inference time and throughput.

Table 5: Phase 2—Inference Time (s/intent) [95% CI] and Throughput (tokens/s)

Size	Q2_K Time	Q4_K_M Time	Q8_0 Time	Q2_K Tok/s	Q8_0 Tok/s
0.5B	4.31 [3.91, 4.70]	4.67 [4.30, 5.04]	4.84 [4.45, 5.23]	9.4	7.0
1.5B	7.80 [5.45, 10.14]	7.52 [6.98, 8.06]	5.30 [4.95, 5.65]	5.4	8.9
3.0B	9.47 [8.86, 10.07]	9.44 [8.81, 10.06]	7.66 [7.10, 8.22]	3.8	4.9
7.0B	13.78 [12.74, 14.83]	15.00 [13.87, 16.12]	20.38 [18.94, 21.82]	3.0	1.9

Finding 5: A quantization–speed crossover between compute-bound and bandwidth-bound regimes. At 1.5B and 3B, Q8_0 is the *fastest* variant (5.3 s and 7.7 s, 29–19% faster than Q4_K_M) despite the larger weight files: 8-bit weights feed directly into CPU SIMD pipelines, whereas K-quant 4- and 2-bit blocks require runtime dequantization. At 7B the relationship inverts—Q8_0 becomes the *slowest* variant (20.4 s vs. 15.0 s for Q4_K_M)—because its 11.4 GB resident working set (Section 5.8) saturates memory bandwidth, and the bandwidth penalty of moving twice the bytes outweighs the dequantization savings. The crossover point between the compute-bound regime (dequantization overhead dominates; favors Q8_0) and the bandwidth-bound regime (byte volume dominates; favors Q4/Q2) lies between 3B and 7B on this hardware class. This is a directly actionable systems result: the fastest quantization level depends on model size relative to the platform’s bandwidth ceiling, and cannot be assumed constant.

Finding 6: 0.5B efficiency ceiling. The 0.5B models show nearly identical inference times across quantization levels (4.3–4.8 s), indicating that at this size, inference is dominated by fixed overhead (tokenization, prompt processing, API round-trip) rather than weight-dependent computation. The wide CI at 1.5B Q2_K ([5.45, 10.14]) reflects occasional generation-length outliers rather than systematic slowness—its median latency is competitive.

5.3 Format Accuracy

Table 6 shows format accuracy (valid JSON output) across configurations.

Table 6: Phase 2—Format Accuracy (%) by Size and Quantization

Size	Q2_K	Q4_K_M	Q8_0	Mean
0.5B	99.5	96.0	100.0	98.5
1.5B	99.5	99.0	96.5	98.3
3.0B	99.0	98.5	99.0	98.8
7.0B	100.0	100.0	100.0	100.0

Format accuracy is generally high across all configurations ($\geq 96.0\%$), with a clear upward trend with model size. All 7B configurations achieve perfect (100.0%) format compliance regardless of quantization level.

Actionable insight: Format compliance is not a differentiator at $\geq 1.5\text{B}$. When selecting between models of different sizes, accuracy, latency, and resource metrics are the primary decision variables; format compliance is essentially guaranteed for models of this scale with the enriched prompt.

5.4 Domain-Level Analysis

Table 7 presents action correct accuracy by domain for selected configurations (full data for all 12 configurations is available in the project repository).

Table 7: Phase 2—Action Correct by Domain (%) [Selected Configurations]

Configuration	Provisioning	Scaling	Conflict	Provisioning Exact Match
0.5B Q2_K	44.3	14.3	11.7	31.2
0.5B Q8_0	40.0	22.9	13.3	31.5
1.5B Q4_K_M	75.7	30.0	13.3	43.7
1.5B Q8_0	75.7	28.6	11.7	44.0
3.0B Q4_K_M	100.0	22.9	8.3	52.4
7.0B Q2_K	98.6	15.7	6.7	53.4
7.0B Q4_K_M	100.0	18.6	6.7	55.2
7.0B Q8_0	100.0	21.4	5.0	56.5

Finding 7: Provisioning performance scales rapidly with size. At 0.5B, models struggle even with Provisioning (40–70% action correct). By 1.5B, accuracy jumps

to ~76%, and by 3B, the domain is essentially solved (100% action correct at Q4_K_M). The exact match metric tells a more nuanced story: even at 7B Q8_0, Provisioning exact match reaches only 56.5%, indicating that while the action is correctly identified, parameter-level accuracy (bandwidth, latency, device density) continues to improve with size.

Finding 8: Scaling and Conflict domains do not benefit from larger models.

Across the 0.5B–7B range, Scaling Request action accuracy fluctuates between 14.3–30.0% with no clear upward trend—indeed the 1.5B models outperform the 7B models on this domain—and Conflict Resolution remains stubbornly at 5.0–13.3%, with a mild *negative* size trend. This plateau suggests that size scaling alone, without architectural innovations or fine-tuning, cannot address the multi-step reasoning required for these domains.

5.5 Complexity Robustness

Table 8 examines exact-match accuracy across the three complexity tiers.

Table 8: Phase 2—Exact Match by Complexity (%)

Configuration	Simple	Complex	Ambiguous	Simple-to-Complex (pp)
0.5B Q2_K	36.2	12.1	5.3	-24.1
0.5B Q8_0	36.4	13.7	6.7	-22.7
1.5B Q4_K_M	42.2	18.2	15.6	-24.0
1.5B Q8_0	44.5	18.7	12.3	-25.8
3.0B Q4_K_M	44.9	18.1	17.4	-26.8
7.0B Q2_K	47.5	19.3	18.7	-28.2
7.0B Q4_K_M	46.7	20.4	21.2	-26.3
7.0B Q8_0	47.7	21.6	20.8	-26.1

The complexity degradation is remarkably consistent across all 12 configurations: Simple-to-Complex exact match drops by 22.7–28.2 percentage points. This uniformity suggests that the complexity distinction captures a fundamental difficulty boundary that is orthogonal to model size and quantization—it reflects the gulf between explicit parameter extraction and implicit multi-step reasoning that current SLM architectures cannot bridge at any tested scale.

Encouraging trend on Ambiguous intents: Unlike the flat Complex tier, Ambiguous-intent accuracy scales markedly with size—from 5–7% at 0.5B to 19–21% at 7B, a 3x improvement. Larger models are substantially better at inferring

implicit priorities from context, even though multi-constraint Conflict reasoning remains out of reach. This decoupling (Ambiguous improves with scale, Conflict does not) localizes the SLM reasoning gap to *combinatorial constraint resolution* rather than *contextual inference* generally.

5.6 Token Efficiency

Table 9 summarizes total token generation across configurations.

Table 9: Phase 2—Token Generation Summary

Size	Q2_K Tokens	Q4_K_M Tokens	Q8_0 Tokens	Tokens/Intent Range
0.5B	8,071	7,730	6,770	34–40
1.5B	8,474	9,783	9,415	42–49
3.0B	7,108	7,189	7,460	36–37
7.0B	8,318	8,343	7,930	40–42

Token generation per intent ranges from 34 (0.5B Q8_0, the most concise) to 49 (1.5B Q4_K_M, the most verbose). The 1.5B tier is systematically the most verbose, which partially explains its latency profile. The larger models (3B–7B) show consistent token counts (36–42 tokens/intent) across quantization levels, indicating stable output verbosity.

Total benchmark time ranges from 14.4 minutes (0.5B Q2_K) to 67.9 minutes (7B Q8_0)—a 4.7x range that has direct operational implications for model selection in time-sensitive deployment scenarios.

5.7 Combined Pareto Analysis

When all 12 Phase 2 configurations are plotted in the accuracy–speed space, a clear Pareto frontier emerges:

- **Frontier point 1:** 0.5B Q8_0 (19.4%, 4.8 s)—best accuracy at the <5 s latency tier
- **Frontier point 2:** 1.5B Q8_0 (25.7%, 5.3 s)—the *value champion*: +6.3 pp over the 0.5B tier for only +0.5 s, at 611 J/intent the most energy-efficient usable configuration
- **Frontier point 3:** 3B Q8_0 (26.8%, 7.7 s) and 3B Q4_K_M (27.3%, 9.4 s)—the mid-range sweet spots
- **Frontier point 4:** 7B Q2_K (29.0%, 13.8 s)—near-maximal accuracy, leveraging aggressive quantization to undercut 7B Q4/Q8 latency

- **Frontier point 5:** 7B Q8_0 (30.5%, 20.4 s)—absolute accuracy leader, suitable when latency budgets allow

Notably, the 1.5B Q8_0 point dominates every 1.5B and 0.5B alternative simultaneously on accuracy, speed, and energy—a rare triple dominance that makes it the default recommendation for constrained deployments.

5.8 Resource Consumption: CPU, Memory, and Energy

This section presents the per-intent resource telemetry—the principal addition of the instrumented re-run. **Table 10** reports measured CPU utilization, peak resident memory, and derived energy for all 12 configurations.

Table 10: Phase 2—Resource Telemetry [mean, 95% CI, n=200]

Configurat ion	CPU (%)	Peak RSS (MB)	Energy (J/intent)	Energy Eff. (Match%/kJ)
0.5B Q2_K	68.8 [67.0, 70.6]	2,605	436 [384, 487]	42.0
0.5B Q4_K_M	69.1 [67.2, 71.1]	575	477 [429, 525]	38.8
0.5B Q8_0	71.6 [70.0, 73.3]	2,166	505 [454, 555]	38.4
1.5B Q2_K	76.9 [75.3, 78.5]	1,372	897 [592, 1,202]	24.6
1.5B Q4_K_M	80.6 [79.5, 81.8]	1,720	850 [780, 920]	30.4
1.5B Q8_0	83.1 [82.2, 84.1]	2,480	611 [566, 656]	42.1
3.0B Q2_K	85.8 [85.1, 86.4]	2,453	1,117 [1,037, 1,198]	21.6
3.0B Q4_K_M	86.1 [85.3, 87.0]	3,323	1,122 [1,039, 1,205]	24.3
3.0B Q8_0	88.4 [87.8, 89.1]	5,225	931 [857, 1,005]	28.8
7.0B Q2_K	91.2 [90.8, 91.6]	8,149	1,719 [1,580, 1,857]	16.9
7.0B Q4_K_M	91.9 [91.4, 92.3]	4,922	1,884 [1,734, 2,034]	15.8
7.0B Q8_0	94.1 [93.8, 94.5]	11,429	2,613 [2,420, 2,806]	11.7

Finding 9: CPU utilization scales with model size—direct evidence of the memory bandwidth bottleneck. Measured system-wide CPU utilization rises

monotonically from 68.8% (0.5B Q2_K) to 94.1% (7B Q8_0), with tight confidence intervals ($\pm 1\text{--}2$ pp) confirming the trend is systematic. Small models cannot keep 16 cores busy: their weight matrices are small enough that cores stall waiting for sequential token dependencies rather than memory transfers. As parameter count grows, weight streaming dominates and utilization asymptotes toward saturation. Within each size class, higher-precision variants (more bytes moved per token) show consistently higher utilization, corroborating the bandwidth interpretation. To our knowledge this is the first published per-intent CPU utilization curve for SLM inference on virtualized edge-class hardware.

Finding 10: Flat-utilization energy models systematically overestimate small models. Energy derived from measured utilization is 15–25% lower for 0.5B configurations than a naive “95% of TDP” model would predict (e.g., 0.5B Q4_K_M: 477 J measured-utilization basis vs. 599 J flat-model basis). For fleet-level 6G capacity planning—where far-edge nodes will predominantly run small models—this bias compounds directly into overprovisioned power and cooling budgets.

Finding 11: Energy efficiency inverts the accuracy ranking. Expressed as accuracy per kilojoule, the ranking is led by 1.5B Q8_0 (42.1 Match%/kJ) and the 0.5B tier (38–42), while the accuracy-leading 7B tier is 2.5–3.6x less efficient (11.7–16.9). The steepness is notable: moving from 1.5B Q8_0 to 7B Q8_0 buys +4.8 pp exact match at 4.3x the energy per intent. Under carbon- or power-capped operation, 1.5B–3B configurations deliver most of the achievable accuracy at a fraction of the joule cost.

Finding 12: Memory footprint, not disk size, is the deployment constraint. Peak RSS ranges from 575 MB (0.5B Q4_K_M) to 11.4 GB (7B Q8_0). The 7B Q8_0 configuration exceeds the 8 GB memory envelope of typical far-edge nodes, whereas 7B Q4_K_M (4.9 GB) fits comfortably—a further argument for Q4_K_M as the default precision. We note one measurement caveat: the custom-built Q2_K and Q8_0 variants (imported GGUF via Modelfile) exhibit higher resident memory than their file sizes alone would suggest, because Ollama materializes imported weights differently from library models; RSS comparisons are therefore most reliable *within* a provisioning method (all Q4_K_M rows use library tags).

6. Discussion

6.1 Deployment Guidelines for 6G Edge

Our results support the following decision framework for SLM selection, now incorporating measured energy and memory:

Deployment Scenario	Recommended Config	Rationale
Maximum accuracy, relaxed latency (>20 s acceptable)	Qwen 2.5 7B, Q8_0	30.5% exact match; requires >11 GB RAM and 2.6 kJ/intent
Balanced accuracy-speed (production default)	Qwen 2.5 7B, Q4_K_M	29.8% at 15.0 s, 4.9 GB RAM, 28% less energy than Q8_0
Memory-constrained edge (<5 GB available)	Qwen 2.5 7B, Q2_K or 3B, Q4_K_M	29.0% / 27.3%; Q2_K trades 0.8 pp for lower footprint
Efficiency sweet spot (<4 GB, fast)	Qwen 2.5 3B, Q4_K_M	27.3% at 9.4 s; 92% of 7B accuracy at 60% of its energy
Energy-capped / sustainability-first	Qwen 2.5 1.5B, Q8_0	25.7% at 5.3 s and 611 J; best Match%/kJ of all 12 configs
Ultra-low-latency (<5 s target)	Qwen 2.5 0.5B, Q8_0	19.4% at 4.8 s; best accuracy under 5 s
Provisioning-only workloads (single domain)	Qwen 2.5 3B, Q4_K_M	100% action correct on Provisioning at 3.3 GB RAM

6.2 Cross-Family Selection

For deployments not restricted to Qwen 2.5, the Phase 1 results offer additional guidance:

- **Qwen 2.5 7B** is the unambiguous accuracy–speed leader at ~8B.
- **Llama 3.1 8B** provides a strong alternative with identical format compliance and competitive speed, suitable when ecosystem compatibility (Llama tooling, fine-tuning infrastructure) is prioritized.
- **Gemma 2 9B** should be considered only when action prediction (45.5%) is the primary metric and latency constraints are relaxed (>30 s acceptable).
- **GLM-4 9B** is not recommended for English-language network orchestration without further prompt adaptation or fine-tuning.
- **DeepSeek-R1 and all reasoning-oriented architectures** should be avoided for CPU-only edge deployment pending resolution of the unbounded token generation issue.

6.3 Limitations and Future Work

Energy measurement basis: Energy is derived from measured CPU utilization x TDP x time rather than direct hardware counters, because RAPL (/sys/class/powercap) is unavailable inside Proxmox KVM guests. The utilization-based estimate removes the dominant error source of flat-TDP models (Finding 10), but absolute joule values retain an estimated $\pm 15\%$ uncertainty. Host-level RAPL or external power metering is planned.

CPU-only scope: All experiments run on CPU (16-core VM). GPU-accelerated inference would shift the speed–accuracy Pareto frontier, potentially changing the relative ranking of models (particularly larger variants that benefit more from GPU parallelism). A GPU-accelerated replication is planned.

Single-prompt design: Results reflect one prompt engineering approach. Systematic prompt variation, few-shot example scaling, and chain-of-thought prompting (for non-reasoning models) may yield different relative rankings. Prompt robustness analysis is a natural extension.

Domain coverage: The 200-intent dataset, while covering three critical domains, cannot represent all 6G intent types. Fault management, performance optimization, and security policy intents remain unaddressed.

No fine-tuning: All models are evaluated zero-shot with the enriched prompt. Fine-tuning on domain-specific intent-to-JSON data could substantially improve

accuracy, particularly for Scaling and Conflict domains. This is a high-priority direction for future work.

Single-pass evaluation: Each intent is evaluated once per configuration. The reported 95% confidence intervals capture across-intent variance ($n = 200$) but not across-run variance; at temperature=0.1 the latter is expected to be small. Multi-seed replication would tighten this further.

7. Conclusion

This paper presented the first systematic, multi-dimensional empirical evaluation of Small Language Models for intent-driven slice lifecycle management in 6G core networks, with per-intent resource telemetry and 95% confidence intervals across 16 model configurations and 3,200+ instrumented inferences. We identified:

1. **Architecture matters more than size at ~8B:** Qwen 2.5 7B outperforms geo-diverse competitors (Llama 3.1, Gemma 2, GLM-4) by 0.5–5.2 pp exact match while being 1.3–2.7x faster, demonstrating that architecture quality dominates over minor parameter count differences at this scale.
2. **CPU utilization scales with model size (69% to 94%):** Direct psutil telemetry shows that small models cannot saturate a 16-core edge node, empirically confirming memory bandwidth as the binding constraint of CPU inference and invalidating flat-utilization energy models, which overestimate small-model energy by 15–25%.
3. **3B is the efficiency sweet spot:** Qwen 2.5 3B at Q4_K_M achieves 92% of 7B accuracy at 40% of the disk footprint, 60% of the energy, and 1.6x the speed, making it the recommended configuration for resource-constrained edge deployments; 1.5B Q8_0 is the energy-efficiency champion (42 Match %/kJ).
4. **Aggressive quantization can outperform larger high-bit models:** 7B Q2_K (29.0%) surpasses 3B Q8_0 (26.8%), and the quantization sensitivity curve is inverted-U shaped with peak vulnerability at 1.5B. Meanwhile Q4_K_M is statistically indistinguishable from Q8_0 at every size (mean gap 0.3 pp), making 4-bit the default recommendation.
5. **The fastest quantization level is size-dependent:** Q8_0 is the fastest variant at 1.5B–3B (dequantization overhead dominates) but the slowest at

7B (bandwidth dominates), locating the compute-to-bandwidth crossover between 3B and 7B on edge-class CPUs—a systems-level result invisible to accuracy-only benchmarks.

6. **Domain difficulty spans an order of magnitude:** Slicing Provisioning is solved at $\geq 3B$ (100% action accuracy), Scaling Request plateaus near 20–30% irrespective of scale, and Conflict Resolution remains near-random (5–13%). Ambiguous-intent accuracy triples with scale while Conflict does not budge, localizing the SLM reasoning gap to combinatorial constraint resolution.
7. **Complexity robustness is a universal weakness:** All configurations lose 23–28 pp exact match from Simple to Complex intents, indicating that current SLMs lack the reasoning depth for multi-constraint, context-dependent intent understanding regardless of scale.
8. **Reasoning models are not edge-ready:** DeepSeek-R1’s catastrophic failure on CPU (0% format, unbounded token generation) serves as a caution that architectural innovations must be validated on deployment-hardware targets, not just GPU benchmarks.

These findings establish a quantitative foundation for SLM selection and configuration in 6G intent-driven network management. They also identify clear directions for improvement: Conflict Resolution and Complex intent understanding require advances beyond simple size scaling, potentially through domain-specific fine-tuning, retrieval-augmented generation, or hybrid systems that combine SLMs with symbolic reasoning modules.

Data Availability

All experimental data (28 JSON result files including the instrumented `resource_v3` re-run), benchmark code (`benchmark_runner_v2.py` with `psutil` telemetry and CI computation), visualization scripts, and 70+ generated figures are available in the project repository (github.com/bottle-kung/empirical-benchmarking-slm-agents-6g-slice-management, branch `v2`). The dataset of 200 annotated intents is provided in `dataset/intents.jsonl`. A comprehensive project timeline is documented in `timeline.md`.

References

- [1] E. Zeydan and Y. Turk, “Recent Advances in Intent-Based Networking: A Survey,” *IEEE Communications Surveys & Tutorials*, vol. 25, no. 4, pp. 3160–3194, 2023.
- [2] ETSI GS ZSM 011 V1.2.1, “Zero-touch Network and Service Management (ZSM); Intent-Driven Closed Loops,” European Telecommunications Standards Institute, 2023.
- [3] 3GPP TS 28.312 V18.0.0, “Management and Orchestration; Intent Driven Management Services for Mobile Networks,” 3rd Generation Partnership Project, Release 18, 2024.
- [4] Y. Zhang, A. Leivadeas, and M. Ibnkahla, “Large Language Models for Intent-Based Networking: Opportunities and Challenges,” *IEEE Network*, vol. 38, no. 5, pp. 222–230, 2024.
- [5] A. Leivadeas and M. Falkner, “A Survey on Intent-Based Networking,” *IEEE Communications Surveys & Tutorials*, vol. 25, no. 1, pp. 648–675, 2023.
- [6] M. Polese, L. Bonati, S. D’Oro, S. Basagni, and T. Melodia, “Understanding O-RAN: Architecture, Interfaces, Algorithms, Security, and Research Challenges,” *IEEE Communications Surveys & Tutorials*, vol. 25, no. 2, pp. 1376–1411, 2023.
- [7] A. Dubey et al., “The Llama 3 Herd of Models,” *arXiv preprint arXiv:2407.21783*, 2024.
- [8] Qwen Team, “Qwen2.5: A Party of Foundation Models,” Alibaba Cloud, Technical Report, 2024. [Online]. Available: <https://qwenlm.github.io/blog/qwen2.5/>
- [9] Gemma Team, “Gemma 2: Improving Open Language Models at a Practical Size,” Google DeepMind, Technical Report, 2024. [Online]. Available: <https://ai.google.dev/gemma>
- [10] GLM Team, “ChatGLM: A Family of Large Language Models from GLM-130B to GLM-4,” Tsinghua University, Technical Report, 2024.
- [11] A. Gholami, S. Kim, Z. Dong, Z. Yao, M. W. Mahoney, and K. Keutzer, “A Survey of Quantization Methods for Efficient Neural Network Inference,” in *Low-Power Computer Vision*, Chapman and Hall/CRC, pp. 291–326, 2022.

- [12] G. Gerganov, “llama.cpp: GGUF K-Quant Implementation,” GitHub repository, 2024. [Online]. Available: <https://github.com/ggerganov/llama.cpp>
- [13] Z. Jiang, Y. Li, J. Chen, X. Ren, and Y. Zhang, “Small Language Models: Survey, Measurements, and Insights,” *arXiv preprint arXiv:2409.15790*, 2024.
- [14] L. Fan, H. Xue, J. Li, Z. Wang, and M. Yang, “A Survey of Small Language Models,” *arXiv preprint arXiv:2410.20011*, 2024.
- [15] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “QLoRA: Efficient Finetuning of Quantized Language Models,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.
- [16] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, “SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models,” in *International Conference on Machine Learning (ICML)*, 2023.
- [17] T. Sanguannam et al., “SLM Agent Evaluation for Intent-Driven Slice Lifecycle Management in 6G — Version 1,” RB-RU Research, Technical Report, 2025.
-

This work was conducted on Proxmox-virtualized CPU-only infrastructure at RB-RU Research. All models are open-weight and publicly available. The benchmarking framework is reproducible with the provided scripts and dataset.